

Enterprise Firewall, DMZ, Suricata IDS and Wazuh SIEM Lab

Advanced Technical Report with Action Log, Risk Register and Evidence Matrix

Prepared by	Pulindu Randiya Atala Vithanage
Programme	Bachelor of Computer Science - Cyber Security
Environment	Oracle VirtualBox home lab
Project date	July 2026

Project purpose

I built this lab to understand how network segmentation, firewall policy, intrusion detection and central log collection work together in a small enterprise-style environment. The focus was on building the controls, generating safe test traffic and proving each result with screenshots.

1. Executive Summary

I built this lab because I wanted to move beyond using security tools one at a time and see how they fit together as a complete monitoring environment. The lab was created in Oracle VirtualBox and used pfSense as the firewall, Windows 10 as an internal LAN client, Ubuntu Server as a DMZ web server, Kali Linux as the testing and Wazuh host, Suricata as the network IDS, and Wazuh as the SIEM platform.

The network was divided into separate LAN and DMZ subnets. I placed an Apache web server in the DMZ, allowed the services it needed, and created a firewall rule that blocked the DMZ from reaching the internal LAN. I then tested the rule from Ubuntu and confirmed the block both from the failed connection attempt and from pfSense firewall logs.

For detection testing, I used Nmap from Kali against the DMZ web server. Suricata was configured with Emerging Threats Open rules and detected the scan traffic as a Priority 1 Web Application Attack event. Finally, I configured pfSense to forward syslog events to the Wazuh manager over UDP port 514. Packet capture proved that pfSense filterlog events reached the Kali/Wazuh host and were forwarded into the Docker network.

One limitation remained at the end of the project: I did not prove that a pfSense firewall event appeared as a Wazuh dashboard alert. The transport path and Wazuh syslog listener were verified, but custom decoding or alert-rule tuning is still required for a complete end-to-end dashboard alert. I have recorded this honestly rather than presenting the integration as fully complete.

This advanced version adds a chronological action log, a troubleshooting decision log, a security control analysis, a risk register and an evidence matrix. I used sequential action IDs instead of inventing exact timestamps for steps where the wall-clock time was not recorded.

Project result at a glance

Control / Test	Status	Evidence
pfSense WAN/LAN/DMZ segmentation	Completed	Interface assignments and IP addressing verified
DMZ Apache web server	Completed	Web page opened from the LAN
DMZ to LAN isolation	Completed	100% packet loss and pfSense block log
Suricata IDS detection	Completed	Priority 1 Web Application Attack alerts
pfSense to Wazuh syslog transport	Validated at transport layer	UDP 514 packets and raw filterlog payload captured
Wazuh dashboard alert for pfSense event	Future work	No matching alert was confirmed in alerts.log/dashboard

2. Objectives and Scope

The main objectives of the lab were:

- Build a three-interface pfSense firewall with separate WAN, LAN and DMZ networks.
- Place an Ubuntu Apache server in the DMZ and keep the internal LAN isolated from it.
- Apply least-privilege firewall rules so the DMZ could use only the services required for the lab.
- Generate controlled reconnaissance traffic with Nmap and observe IDS detection in Suricata.
- Forward pfSense security logs to Wazuh using syslog over UDP port 514.
- Troubleshoot each layer with evidence instead of assuming that a service was working.

Ethical testing scope

All tests were performed inside my own VirtualBox lab using private IP addresses and systems that I configured. The Nmap scans and ICMP tests were only directed at lab machines. No third-party system or public target was tested.

3. Lab Design and Network Architecture

The design uses pfSense as the central control point between the internal LAN and a separate DMZ. This made it possible to test both normal access and blocked traffic in a controlled way. Kali was connected to the LAN for scanning and also hosted the existing Wazuh Docker deployment.

System	Role	Interface / Network	IP address
pfSense	Firewall and router	WAN / em0	10.0.2.15/24
pfSense	LAN gateway	LAN / em1	192.168.10.1/24
pfSense	DMZ gateway	DMZ / em2	192.168.20.1/24
Windows 10	Internal client	LAN_NET	192.168.10.100
Kali Linux	Security testing + Wazuh host	LAN_NET / eth1	192.168.10.101
Ubuntu Server	DMZ Apache server	DMZ_NET / enp0s3	192.168.20.100

VirtualBox NAT / Internet

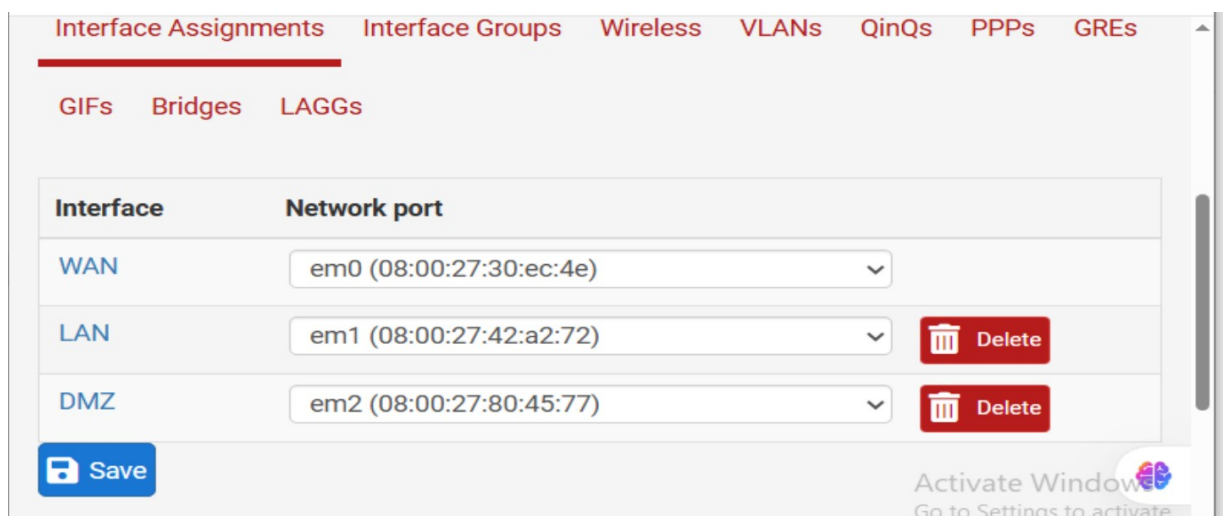
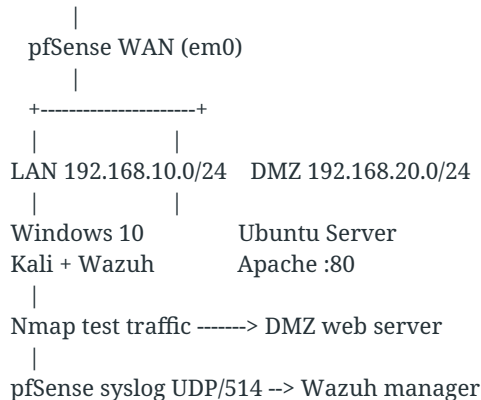
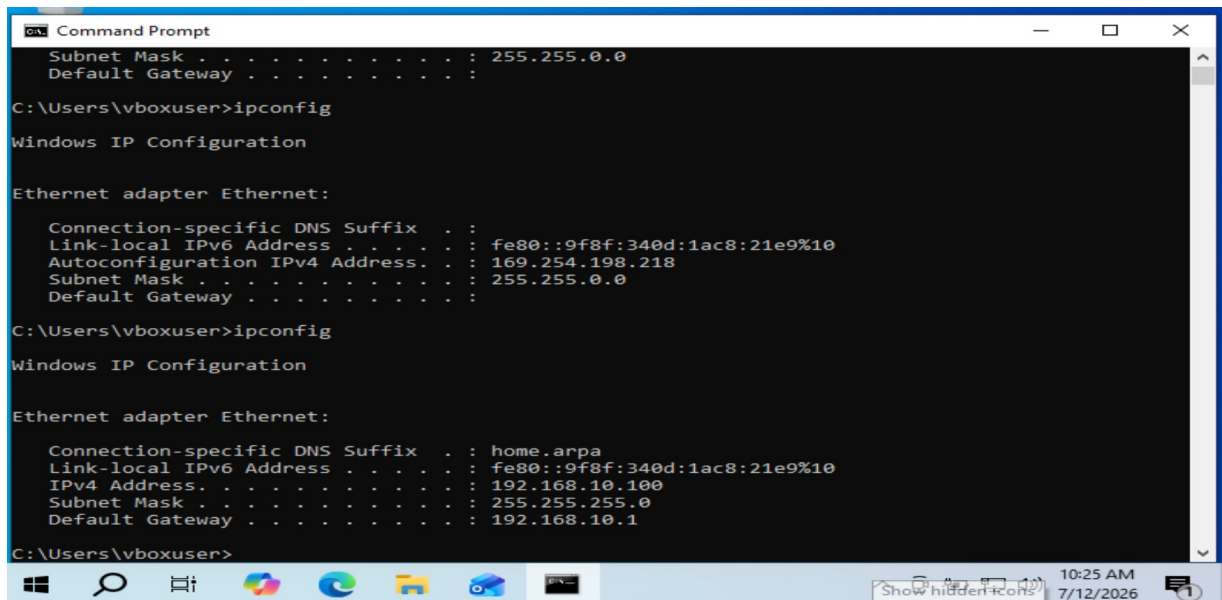


Figure 1. pfSense interface assignments showing em0 as WAN, em1 as LAN and em2 as DMZ.

4. Firewall and DMZ Implementation

4.1 LAN addressing and connectivity

After enabling DHCP on the pfSense LAN, the Windows client received 192.168.10.100/24 with 192.168.10.1 as its default gateway. This confirmed that the LAN side of the firewall was assigning addresses correctly.



```

C:\Users\vboxuser>ipconfig

Windows IP Configuration

Ethernet adapter Ethernet:

    Connection-specific DNS Suffix  . : 
    Link-local IPv6 Address . . . . . : fe80::9f8f:340d:1ac8:21e9%10
    Autoconfiguration IPv4 Address. . . : 169.254.198.218
    Subnet Mask . . . . . : 255.255.0.0
    Default Gateway . . . . . : 

C:\Users\vboxuser>ipconfig

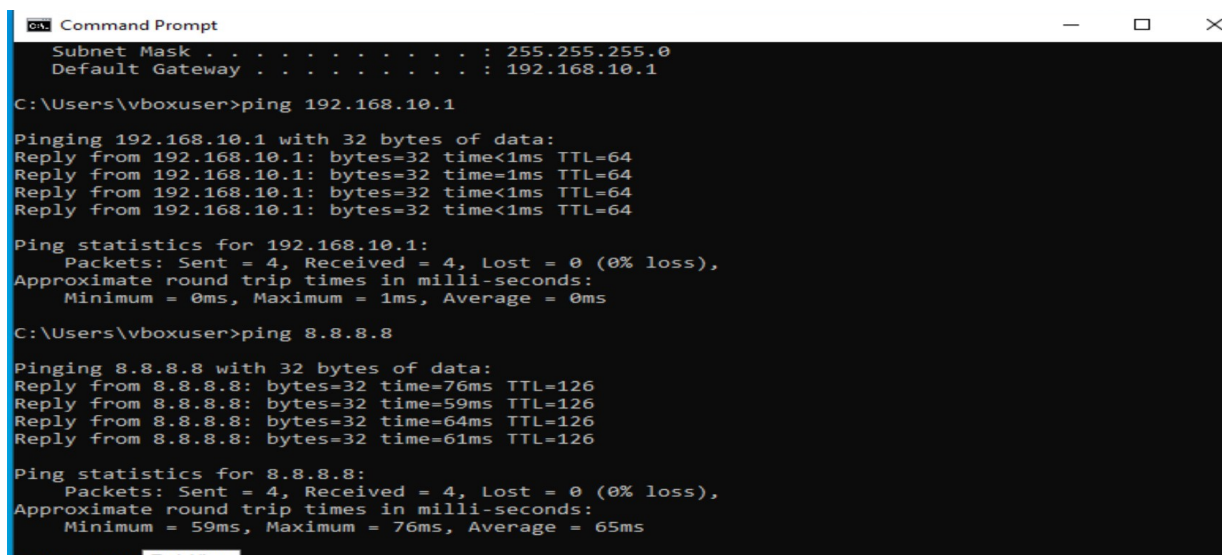
Windows IP Configuration

Ethernet adapter Ethernet:

    Connection-specific DNS Suffix  . : home.arpa
    Link-local IPv6 Address . . . . . : fe80::9f8f:340d:1ac8:21e9%10
    IPv4 Address. . . . . : 192.168.10.100
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : 192.168.10.1
  
```

Figure 2. Windows LAN client receiving 192.168.10.100 with pfSense 192.168.10.1 as the default gateway.

I then tested the local gateway and external connectivity. The Windows client successfully reached both 192.168.10.1 and 8.8.8.8 with 0% packet loss. This was useful because it proved that the client, pfSense LAN interface and outbound path were working before I started adding the DMZ.



```

C:\Users\vboxuser>ping 192.168.10.1

Pinging 192.168.10.1 with 32 bytes of data:
Reply from 192.168.10.1: bytes=32 time<1ms TTL=64
Reply from 192.168.10.1: bytes=32 time=1ms TTL=64
Reply from 192.168.10.1: bytes=32 time<1ms TTL=64
Reply from 192.168.10.1: bytes=32 time<1ms TTL=64

Ping statistics for 192.168.10.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 1ms, Average = 0ms

C:\Users\vboxuser>ping 8.8.8.8

Pinging 8.8.8.8 with 32 bytes of data:
Reply from 8.8.8.8: bytes=32 time=76ms TTL=126
Reply from 8.8.8.8: bytes=32 time=59ms TTL=126
Reply from 8.8.8.8: bytes=32 time=64ms TTL=126
Reply from 8.8.8.8: bytes=32 time=61ms TTL=126

Ping statistics for 8.8.8.8:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 59ms, Maximum = 76ms, Average = 65ms
  
```

Figure 3. Connectivity test from Windows to the pfSense gateway and 8.8.8.8.

4.2 DMZ web server

The Ubuntu Server VM was attached to the VirtualBox internal network named DMZ_NET. The interface initially did not receive a usable address because of a Netplan configuration problem. I

corrected the interface configuration so enp0s3 used DHCPv4, applied the Netplan configuration, and Ubuntu received 192.168.20.100 from pfSense.

Apache2 was installed and started on the Ubuntu server. From the Windows LAN client, I opened <http://192.168.20.100> and received the Apache2 default page. This confirmed that the web service was running and that the allowed LAN-to-DMZ path was functional.

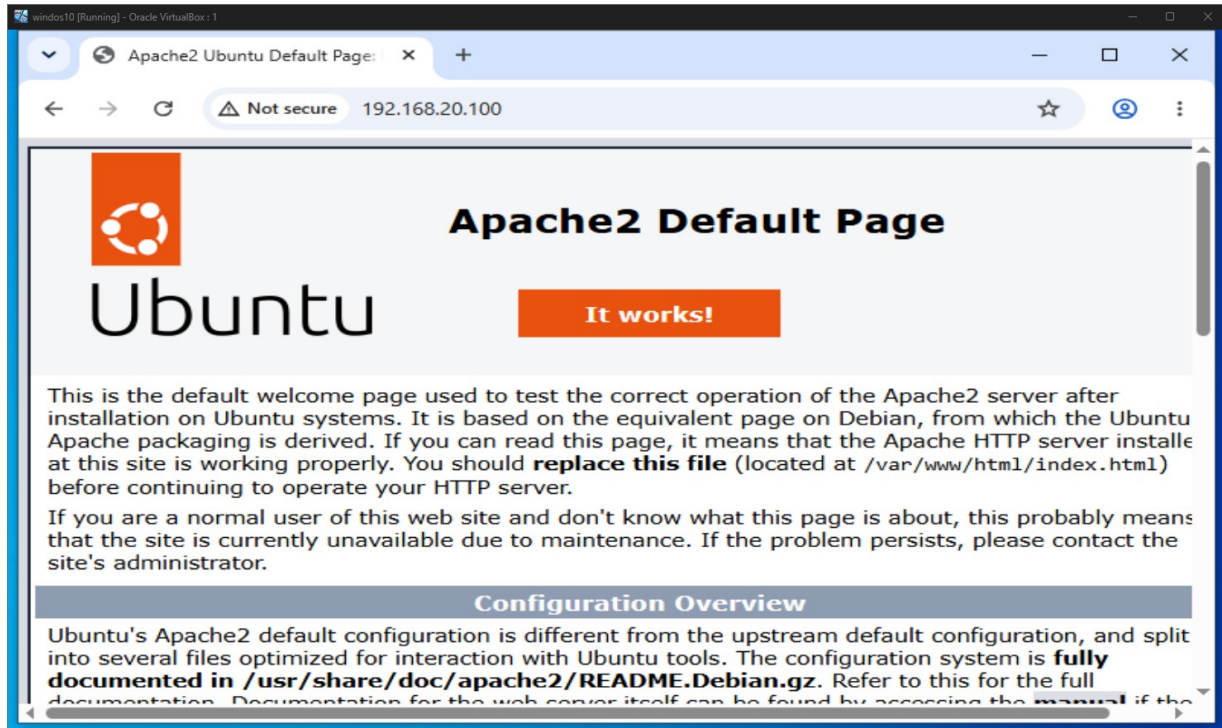


Figure 4. Apache2 default page hosted on the Ubuntu DMZ server at 192.168.20.100.

4.3 DMZ firewall policy

The DMZ rules were written so that the isolation rule was placed above the allow rules. pfSense interface rules are evaluated from top to bottom and use the first matching rule, so the position of a restrictive rule matters [1]. In this lab, DMZ traffic to the LAN was blocked first, while required services such as web access and DNS were allowed separately.

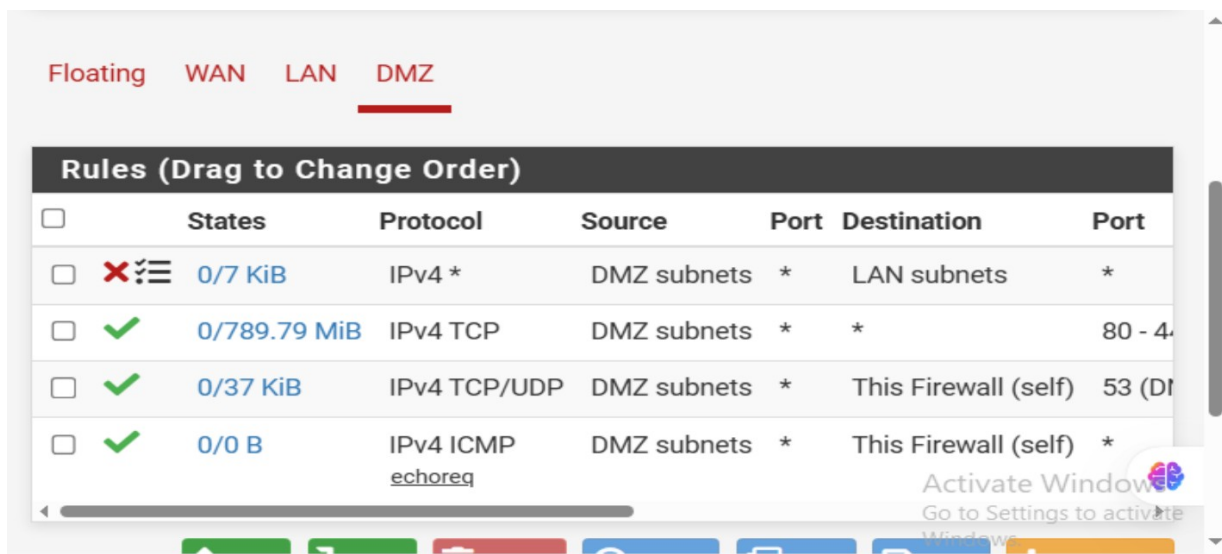


Figure 5. pfSense DMZ firewall rules. The DMZ-to-LAN block rule is placed above the allowed service rules.

To prove that the segmentation worked, I sent ICMP traffic from the Ubuntu DMZ server at 192.168.20.100 to the Windows LAN client at 192.168.10.100. The result was 100% packet loss. The same event appeared in the pfSense firewall log under the rule description 'Block DMZ access to LAN'.

```
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@ubuntu:~$ sudo systemctl status apache2 --no-pager
• apache2.service - The Apache HTTP Server
   Loaded: loaded (/lib/systemd/system/apache2.service; enabled; vendor preset: enabled)
   Active: active (running) since Sun 2026-07-12 08:26:54 UTC; 1min 16s ago
     Docs: https://httpd.apache.org/docs/2.4/
   Main PID: 2353 (apache2)
    Tasks: 55 (limit: 2219)
   Memory: 5.0M
      CPU: 36ms
   CGroup: /system.slice/apache2.service
           └─2353 /usr/sbin/apache2 -k start
             └─2354 /usr/sbin/apache2 -k start
               └─2355 /usr/sbin/apache2 -k start

Jul 12 08:26:54 ubuntu systemd[1]: Starting The Apache HTTP Server...
Jul 12 08:26:54 ubuntu apache2[2352]: AH00558: apache2: Could not reliably determine the se...essage
Jul 12 08:26:54 ubuntu systemd[1]: Started The Apache HTTP Server.
Hint: Some lines were ellipsized, use -l to show in full.
ubuntu@ubuntu:~$ ping -c 4 192.168.10.100
PING 192.168.10.100 (192.168.10.100) 56(84) bytes of data.

--- 192.168.10.100 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3078ms

ubuntu@ubuntu:~$
```

Figure 6. Ubuntu DMZ server failing to reach the Windows LAN client, confirming network isolation.

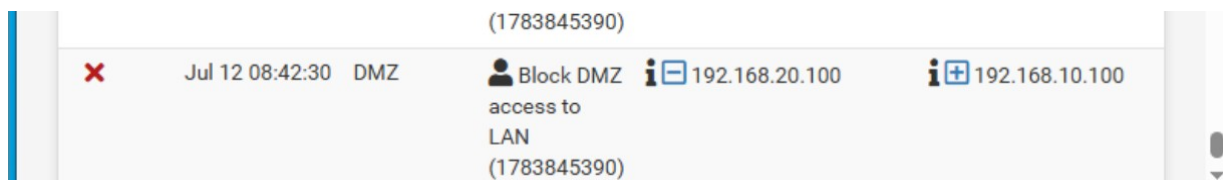


Figure 7. pfSense firewall log showing the DMZ-to-LAN block from 192.168.20.100 to 192.168.10.100.

Security result

The DMZ server could provide its web service to the LAN, but it could not initiate traffic into the internal LAN. This is the main segmentation control I wanted to prove in the firewall part of the project.

5. IDS Testing with Suricata

5.1 Rule update and IDS mode

Suricata was installed on pfSense and used as an intrusion detection system. I enabled the Emerging Threats Open rules and confirmed that the rule update completed successfully.

Suricata supports signature-based alerting, and its alert records contain information about the signature that matched [3].

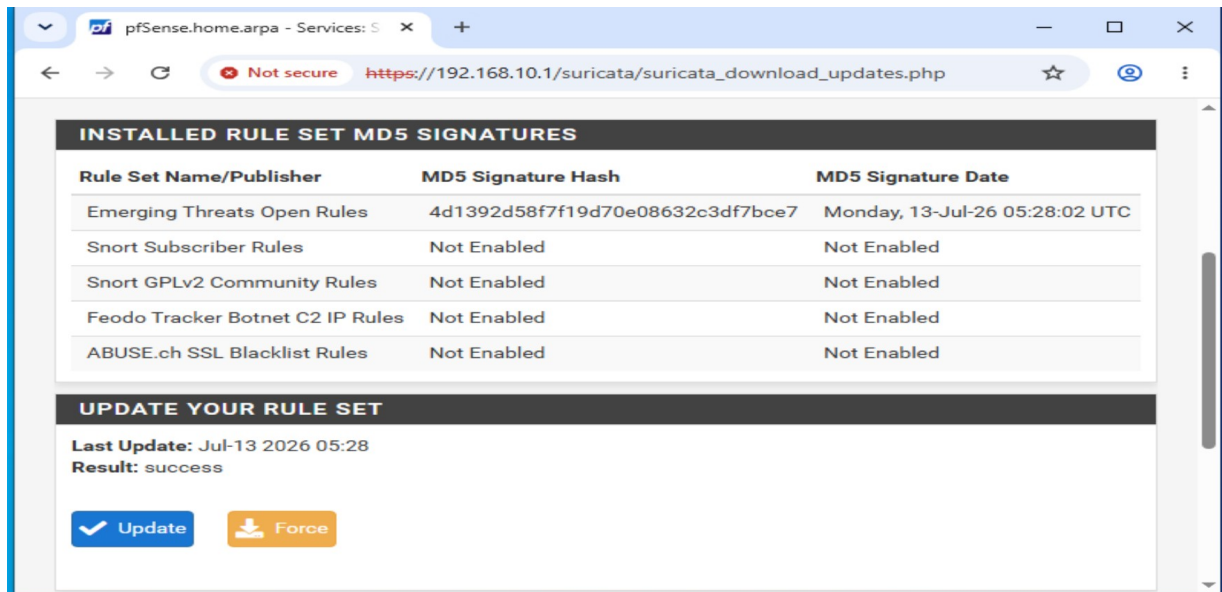


Figure 8. Emerging Threats Open rules successfully downloaded and updated in the pfSense Suricata package.

For this lab I kept Suricata in IDS mode rather than enabling automatic blocking. I wanted to observe and document the detection first. This also made the result easier to separate from the pfSense firewall block policy.

5.2 Controlled reconnaissance test

From Kali, I first ran a service version scan against the DMZ host. Nmap identified TCP port 80 as open and reported Apache httpd 2.4.52 on Ubuntu. I later generated a stronger scan using SYN scanning, service detection, operating system detection and a full port range. The purpose was not exploitation; it was to create recognizable reconnaissance traffic for the IDS.

```
nmap -sV 192.168.20.100
sudo nmap -sS -sV -O -T4 -p- 192.168.20.100
```

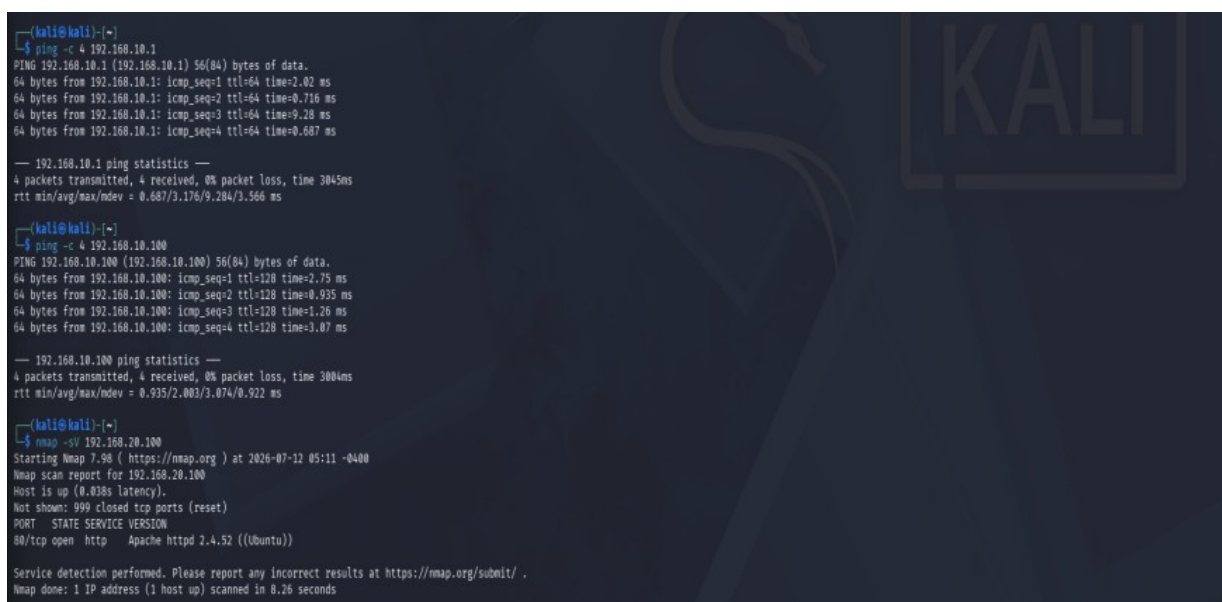
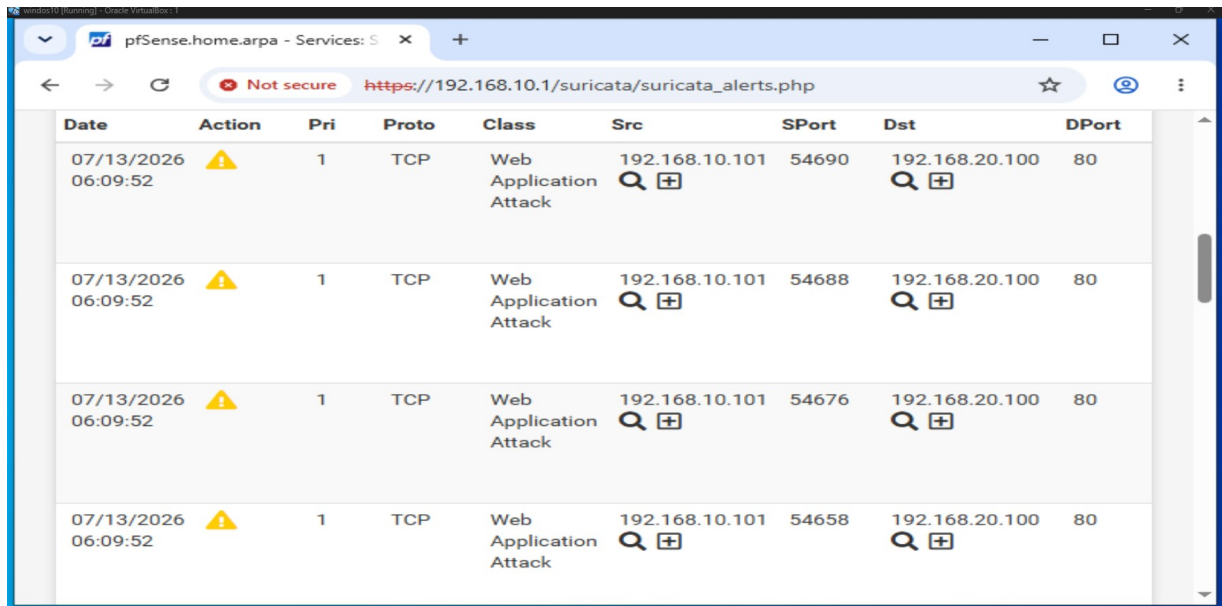


Figure 9. Kali reaching the lab networks and identifying Apache on TCP port 80 of the DMZ server.

5.3 Suricata detection result

After the scan, the Suricata alert view showed multiple Priority 1 TCP alerts classified as 'Web Application Attack'. The source was the Kali LAN address 192.168.10.101 and the destination was the Ubuntu DMZ server 192.168.20.100 on port 80.



Date	Action	Pri	Proto	Class	Src	SPort	Dst	DPort
07/13/2026 06:09:52	⚠	1	TCP	Web Application Attack	192.168.10.101	54690	192.168.20.100	80
07/13/2026 06:09:52	⚠	1	TCP	Web Application Attack	192.168.10.101	54688	192.168.20.100	80
07/13/2026 06:09:52	⚠	1	TCP	Web Application Attack	192.168.10.101	54676	192.168.20.100	80
07/13/2026 06:09:52	⚠	1	TCP	Web Application Attack	192.168.10.101	54658	192.168.20.100	80

Figure 10. Suricata Priority 1 alerts generated from Kali 192.168.10.101 to the DMZ web server 192.168.20.100:80.

This result proves that Suricata detected suspicious scan-related traffic in the lab. It does not prove that the Apache server was compromised. I have kept that distinction clear because an IDS alert is evidence of a rule match, not automatic evidence of a successful attack.

6. Wazuh SIEM and pfSense Syslog Integration

6.1 Wazuh manager environment

I used an existing Wazuh single-node Docker deployment on Kali. The environment included the Wazuh dashboard, indexer and manager containers. All three services were running, and the manager container published UDP port 514. Wazuh documents syslog collection as a method for receiving logs from devices such as firewalls, switches and routers [4].

```

kali@kali:~$ cd ~/wazuh-docker/single-node
kali@kali:~/wazuh-docker/single-node$ sudo systemctl -w vm_max_map_count=262144
vm_max_map_count + 262144
kali@kali:~/wazuh-docker/single-node$ docker compose up -d
[*] Running 3/3
✓ Container single-node-wazuh-indexer-1 Running
✓ Container single-node-wazuh-manager-1 Running
✓ Container single-node-wazuh-dashboard-1 Running
kali@kali:~/wazuh-docker/single-node$ docker compose ps
NAME                                IMAGE                                COMMAND                                SERVICE    CREATED   STATUS    PORTS
single-node-wazuh-dashboard-1       wazuh/wazuh-dashboard:4.14.6       "/entrypoint.sh"                      wazuh.dashboard    2 days ago Up 14 minutes 443/tcp, 0.0.0.0:443->5601/tcp, [::]:443->5601/tcp
single-node-wazuh-indexer-1         wazuh/wazuh-indexer:4.14.6         "/entrypoint.sh open-"                wazuh.indexer      2 days ago Up 14 minutes 0.0.0.0:9200->9200/tcp, [::]:9200->9200/tcp
single-node-wazuh-manager-1         wazuh/wazuh-manager:4.14.6         "/init"                                wazuh.manager      2 days ago Up 14 minutes 0.0.0.0:1514-1515->1514-1515/tcp, [::]:1514-1515->1514-1515/tcp, 0.0.0.0:514->514/udp, [::]:514->514/udp
kali@kali:~/wazuh-docker/single-node$ ip -s a show eth1
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 192:168:10:101/24 brd 192:168:10:255 scope global dynamic noprefixroute eth1
        valid_lft 3658sec preferred_lft 3658sec
kali@kali:~/wazuh-docker/single-node$

```

Figure 11. Wazuh single-node Docker services running on Kali, with the Wazuh manager exposing UDP 514.

6.2 Syslog listener configuration

The Wazuh manager was configured with a dedicated syslog remote block. The listener used UDP port 514 and only allowed the pfSense LAN address 192.168.10.1. This follows the Wazuh remote

configuration model, where the connection type, port, protocol and allowed sender addresses are defined in the manager configuration [5].

```
<remote>
<connection>syslog</connection>
<port>514</port>
<protocol>udp</protocol>
<allowed-ips>192.168.10.1</allowed-ips>
</remote>
```

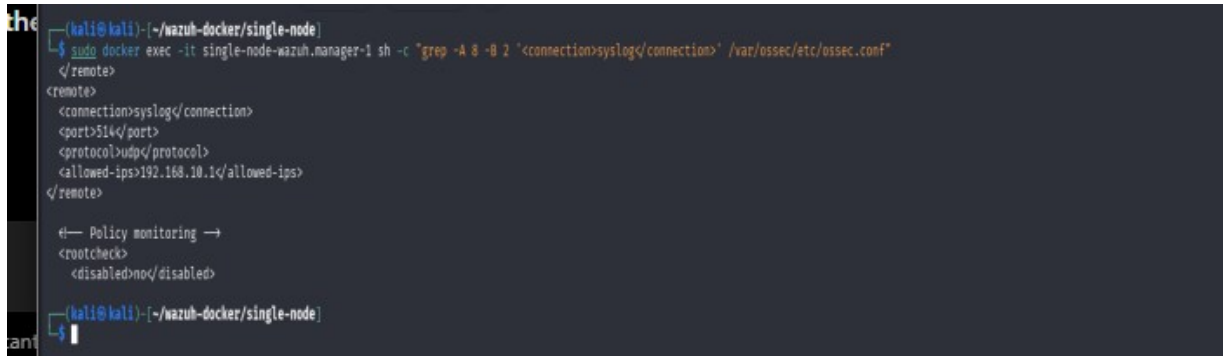


Figure 12. Running Wazuh manager configuration showing the UDP 514 syslog listener and the allowed pfSense IP.

6.3 Proving the log transport path

This was the most difficult part of the project. At first, the Wazuh alert search was blank and I could not see automatic pfSense syslog packets. Rather than assuming the SIEM was broken, I separated the problem into layers. A direct logger test from pfSense proved that UDP 514 could reach Kali. I then returned to the pfSense web interface, disabled remote logging once, saved the configuration, enabled it again with one remote server entry, selected the required log contents and saved the settings.

After that change, tcpdump on Kali captured automatic syslog traffic from 192.168.10.1:514 to 192.168.10.101:514. The packet capture also showed Docker forwarding toward 172.18.0.2:514. Netgate documents remote syslog servers as IP or hostname destinations with an optional UDP port, with 514 used as the default syslog port [2].

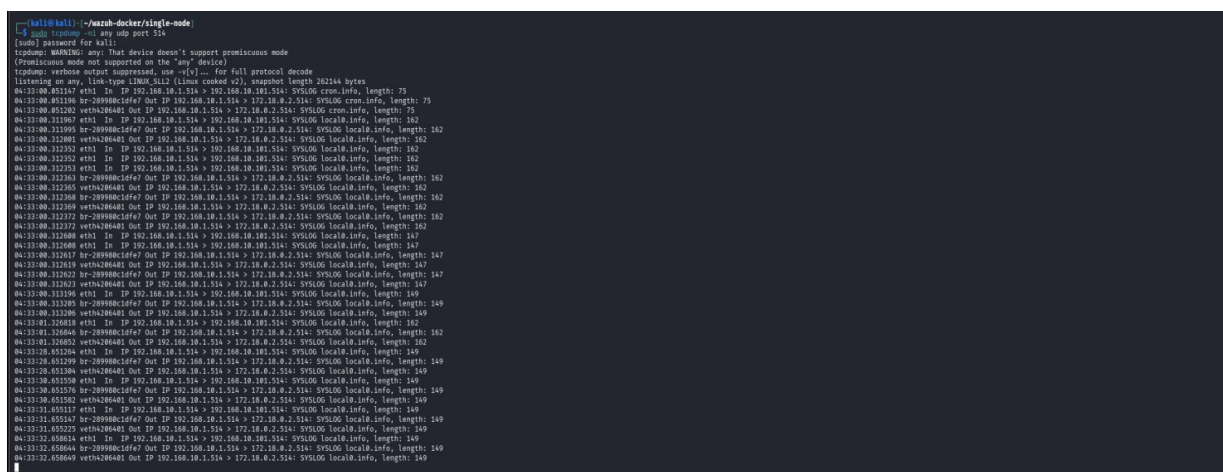


Figure 13. tcpdump proving automatic pfSense syslog traffic reaching the Kali/Wazuh host on UDP 514 and entering the Docker network.

To verify that the traffic was a real firewall event rather than an empty test message, I captured one packet in ASCII mode. The payload contained the pfSense filterlog record, the action 'block',

the source 192.168.20.100 and the destination 192.168.10.100. This directly matched the DMZ-to-LAN firewall test.

```

kali@kali:~/wazuh-docker/single-node
$ sudo tcpdump -i -i eth1 udp port 514 -c 1
tcpdump: verbose output suppressed, use -v[]... for full protocol decode
listening on eth1, link-type EN10MB (Ethernet), snapshot length 262144 bytes
04:49:31.872284 IP 192.168.10.1514 > 192.168.10.101: SYSLOG local0.info, length: 149
E...*...@.....
.....
.....<134>Jul 13 00:49:31 filterlog[56206]: 96,,,1783845390,en2,match,block,in,4,0*0,,64,30645,0,DF,1,icmp,84,192.168.20.100,192.168.10.100,request,12,164
1 packet captured
1 packet received by filter
0 packets dropped by kernel

```

Figure 14. Readable pfSense filterlog syslog payload containing the blocked DMZ-to-LAN event.

What was proven

pfSense generated a firewall log, sent it over UDP 514, Kali received it, and the packet entered the Wazuh Docker path. The running Wazuh manager configuration also contained the expected syslog listener.

What was not proven

A matching pfSense firewall alert was not confirmed in Wazuh alerts.log or the Wazuh dashboard. The final parsing and alert-correlation stage therefore remains future work. The report does not claim an end-to-end Wazuh dashboard alert.

7. Troubleshooting and Lessons Learned

The troubleshooting work became one of the most useful parts of the project. Several issues were caused by configuration state rather than by a failed security product.

7.1 Ubuntu did not receive a DMZ address

The Ubuntu interface was attached to DMZ_NET but initially had no correct IPv4 address. I checked the interface name, corrected the Netplan configuration for enp0s3, enabled DHCPv4 and applied the configuration. Once Ubuntu received 192.168.20.100 from pfSense, I could continue with Apache and firewall testing.

7.2 Remote syslog looked configured but produced no packets

The pfSense remote logging page contained the Wazuh server address, but early packet captures showed zero UDP 514 packets. A direct remote logger test was important because it proved the network path from pfSense to Kali before I changed Wazuh. Re-saving the remote logging configuration cleanly through the pfSense web interface then produced automatic syslog traffic.

7.3 A blank Wazuh alert search did not mean the network path failed

At first I searched alerts.log and found no matching pfSense alert. The packet captures later showed that the log transport itself was working. This taught me to troubleshoot a SIEM pipeline in layers: event generation, network transport, listener configuration, decoding, rules and finally dashboard presentation.

7.4 Important technical lessons

- Firewall rule order matters. A restrictive rule must be placed where it will match before a broader allow rule.
- Segmentation should be tested with real traffic and confirmed in firewall logs.
- An IDS alert means a detection rule matched; it does not automatically mean a system was compromised.
- Packet capture is useful for separating a network transport problem from a SIEM parsing or alerting problem.
- A security report should clearly state the difference between completed controls and unfinished integration work.

8. Results and Security Assessment

The core security goals of the lab were achieved. The internal network and DMZ were separated by pfSense, the DMZ web service remained reachable from the LAN, and DMZ-initiated traffic to the LAN was blocked and logged. Suricata successfully detected the controlled Nmap reconnaissance activity, and the pfSense firewall events were transported to the Wazuh host through the configured UDP 514 syslog path.

The strongest outcome was not a single screenshot or alert. It was the ability to trace one test across multiple layers: Ubuntu generated blocked traffic, pfSense enforced and logged the rule, syslog transported the event, and packet capture verified the exact filterlog payload at the Wazuh host. This gave me a practical view of how network security monitoring is built and troubleshot.

The remaining limitation is Wazuh alert normalization for the pfSense event. The next technical improvement would be to test the raw event with Wazuh log testing tools, confirm the matched decoder, and add or tune a local rule so the specific firewall block is indexed and presented as a meaningful dashboard alert.

9. Conclusion

This project started as a basic firewall lab and developed into a small enterprise-style monitoring environment. I configured three network zones, built a DMZ web server, enforced segmentation, generated safe reconnaissance traffic, detected it with Suricata and connected pfSense logging to a Wazuh SIEM infrastructure.

The project also improved my troubleshooting approach. Instead of repeatedly changing random settings, I learned to prove each layer with the right evidence: IP configuration for addressing, ping for reachability, firewall logs for policy enforcement, Nmap for service discovery, Suricata alerts for IDS detection, tcpdump for syslog transport and Wazuh configuration checks for the SIEM listener.

The Wazuh dashboard alert stage still needs further decoder or rule tuning, but the completed work already demonstrates practical skills in network segmentation, firewall administration, Linux networking, intrusion detection, SIEM integration and technical documentation. For a cybersecurity portfolio, I believe the value of this lab is that the results are supported by real tests and the unfinished part is documented honestly.

Skills Demonstrated

- pfSense firewall configuration and multi-interface network design
- LAN and DMZ segmentation using VirtualBox internal networks
- Linux networking and Ubuntu Netplan troubleshooting
- Apache web server deployment in a DMZ
- Firewall rule design, logging and validation
- Nmap reconnaissance and service version detection
- Suricata IDS configuration and Emerging Threats Open rule updates
- Wazuh single-node Docker operation and syslog listener configuration
- Syslog transport over UDP 514 and tcpdump packet analysis
- Evidence-based troubleshooting and security report writing

10. References

- [1] Netgate. Firewall Fundamentals - pfSense Documentation.
<https://docs.netgate.com/pfsense/en/latest/firewall/fundamentals.html> (accessed 13 July 2026).
- [2] Netgate. Remote Logging with Syslog - pfSense Documentation.
<https://docs.netgate.com/pfsense/en/latest/monitoring/logs/remote.html> (accessed 13 July 2026).
- [3] Open Information Security Foundation. Suricata User Guide and EVE Alert Documentation.
<https://docs.suricata.io/> (accessed 13 July 2026).
- [4] Wazuh. Configuring syslog on the Wazuh server - Log data collection.
<https://documentation.wazuh.com/current/user-manual/capabilities/log-data-collection/syslog.html> (accessed 13 July 2026).
- [5] Wazuh. remote - Local configuration (ossec.conf).
<https://documentation.wazuh.com/current/user-manual/reference/ossec-conf/remote.html> (accessed 13 July 2026).
- [6] MITRE. Network Service Discovery, Technique T1046 - Enterprise ATT&CK.
<https://attack.mitre.org/techniques/T1046/> (accessed 13 July 2026).
- [7] Nmap Project. Nmap Reference Guide and Network Discovery/Security Auditing Documentation. <https://nmap.org/book/man.html> (accessed 13 July 2026).

Appendix A - Advanced Security Analysis

This appendix looks at the lab as a security architecture rather than as a list of tools. I grouped the controls by what they were designed to do and then mapped the controlled Nmap activity to a recognised ATT&CK discovery technique.

A.1 Security control model

Control type	Implementation	Purpose	Validation
Preventive	pfSense DMZ-to-LAN block rule	Stops DMZ-originated traffic from reaching the internal LAN.	Validated by failed ICMP test and pfSense filterlog evidence.
Preventive / least privilege	DMZ DNS and web allow rules	Allows the DMZ server to resolve names and obtain web-based updates without granting general LAN access.	Ubuntu apt update worked while DMZ-to-LAN remained blocked.
Detective	Suricata IDS with ET Open rules	Inspects network traffic and raises alerts when signatures match.	Priority 1 Web Application Attack alerts were recorded for Kali-to-DMZ traffic.
Monitoring	pfSense remote syslog to Wazuh	Copies firewall events to a central monitoring host.	UDP 514 packet capture showed automatic pfSense syslog delivery.
Validation	Nmap and controlled ICMP tests	Generates known lab traffic so firewall and IDS behaviour can be tested safely.	Open TCP/80 discovery, Suricata alerts and DMZ block logs were produced.
Troubleshooting	tcpdump and Wazuh configuration checks	Separates transport problems from parsing/rule problems.	Raw filterlog payload reached the Wazuh host, but no matching dashboard alert was confirmed.

A.2 Detection and ATT&CK context

The Nmap activity in this project was a controlled discovery test. Nmap is designed for network exploration and security auditing [7]. MITRE ATT&CK describes T1046 Network Service Discovery as activity used to discover services running on remote systems and includes network scanning behaviour in its detection context [6]. The scan against 192.168.20.100 therefore provides a reasonable lab mapping to T1046.

This mapping is contextual, not a claim that Kali was a real adversary. I generated the traffic myself inside the isolated lab to test detection. The useful security result is that the IDS produced an alert from a known source and destination, allowing the event to be compared with the test I intentionally performed.

Technique	Name	Lab activity	Test method	Observed evidence
T1046	Network Service Discovery	Kali 192.168.10.101 scanned Ubuntu 192.168.20.100	nmap -sV and full-port SYN/version/OS scan	Suricata Priority 1 alerts to destination TCP/80

A.3 Event chain reconstructed from evidence

The lab produced an end-to-end event chain that can be reconstructed from the screenshots and packet capture. The firewall and IDS portions were fully validated, while the final Wazuh alert correlation stage remained incomplete.

Stage	Action	Evidence / interpretation
1	Ubuntu generated ICMP traffic from the DMZ to the Windows LAN client.	Source 192.168.20.100 -> destination 192.168.10.100.
2	pfSense matched the DMZ-to-LAN block rule.	The connection failed with 100% packet loss.
3	pfSense filterlog recorded the blocked event.	Firewall log showed source and destination addresses for the block.
4	pfSense remote logging sent syslog over UDP 514.	tcpdump captured 192.168.10.1:514 -> 192.168.10.101:514.
5	The Wazuh Docker path received forwarded UDP 514 traffic.	Packet capture showed traffic continuing toward 172.18.0.2:514.
6	The raw syslog payload was inspected in ASCII mode.	The packet contained filterlog, block, 192.168.20.100 and 192.168.10.100.
7	Wazuh alert search did not return a confirmed pfSense alert.	Transport was proven; decoder/rule/dashboard correlation remains future work.

Appendix B - Detailed Action Log

The following action log records the build and troubleshooting sequence. Exact wall-clock time was not recorded for every step, so I used sequential action IDs rather than creating artificial timestamps. Commands are included only where they were part of the actual lab work.

ID	Phase	Action / command	Reason	Evidence / result detail	Status
AL-01	Planning	Defined a three-zone lab: WAN, LAN and DMZ.	Create a simple enterprise-style segmentation model.	Topology design and later interface evidence.	Completed
AL-02	VirtualBox	Configured pfSense Adapter 1 as NAT, Adapter 2 as LAN_NET and Adapter 3 as DMZ_NET.	Create separate virtual network paths.	pfSense interface assignment evidence.	Completed
AL-03	pfSense	Assigned em0=10.0.2.15/24 WAN, em1=192.168.10.1/24 LAN and em2=192.168.20.1/24 DMZ.	Establish firewall gateways for each zone.	Figure 1 / interface assignments.	Completed
AL-04	pfSense DHCP	Enabled LAN DHCP range 192.168.10.100-192.168.10.200.	Provide LAN addressing through pfSense.	Windows received 192.168.10.100.	Completed
AL-05	pfSense DHCP	Enabled DMZ DHCP range 192.168.20.100-192.168.20.200.	Provide isolated DMZ addressing.	Ubuntu received 192.168.20.100.	Completed
AL-06	Windows LAN	Checked IP configuration and default gateway.	Confirm DHCP and LAN gateway operation.	192.168.10.100/24, gateway 192.168.10.1.	Validated
AL-07	Windows LAN	Pinged 192.168.10.1 and 8.8.8.8.	Verify local gateway and outbound connectivity.	0% packet loss in both tests.	Validated
AL-08	Ubuntu DMZ	Identified enp0s3 and corrected Netplan DHCP configuration.	Fix missing DMZ IPv4 address.	netplan set network.ethernets.enp0s3={dhcp4: true, dhcp6: false}.	Completed
AL-09	Ubuntu DMZ	Applied Netplan and confirmed 192.168.20.100/24.	Verify Ubuntu joined DMZ_NET.	Ubuntu interface evidence.	Validated
AL-10	Ubuntu DMZ	Installed Apache2 and confirmed the service was active.	Create a test service in the DMZ.	systemctl status apache2 showed active (running).	Completed
AL-11	Windows -> DMZ	Opened http://192.168.20.100.	Confirm LAN can reach the DMZ web service.	Apache2 default page displayed.	Validated
AL-12	pfSense policy	Created Block DMZ access to LAN rule above allow rules and enabled logging.	Prevent DMZ-initiated access into the internal LAN.	DMZ rule screenshot and firewall log.	Completed
AL-13	pfSense policy	Allowed DMZ DNS to This Firewall on TCP/UDP 53.	Permit controlled name resolution.	DMZ rule screenshot.	Completed

ID	Phase	Action / command	Reason	Evidence / result detail	Status
AL-14	pfSense policy	Allowed required outbound HTTP/HTTPS traffic from the DMZ.	Permit package updates without broad LAN access.	DMZ rule screenshot; apt update worked.	Completed
AL-15	Ubuntu test	Ran ping -c 4 192.168.10.100 from the DMZ.	Test the segmentation control.	100% packet loss.	Validated
AL-16	pfSense log	Reviewed Firewall Logs after the blocked ping.	Confirm the firewall, not a host issue, caused the denial.	Block DMZ access to LAN; 192.168.20.100 -> 192.168.10.100.	Validated
AL-17	Suricata	Installed the Suricata pfSense package.	Add network intrusion detection capability.	Suricata service/package evidence.	Completed
AL-18	Suricata	Enabled Emerging Threats Open rules and updated the ruleset.	Provide current community IDS signatures.	Rule update result: success.	Completed
AL-19	Suricata	Added Suricata on LAN/em1 and kept blocking disabled.	Operate in IDS-only mode for safe observation.	LAN IDS monitoring interface shown.	Completed
AL-20	Suricata	Enabled emerging-scan.rules category.	Improve scan-related detection coverage.	Category configuration performed.	Completed
AL-21	Kali scan	Ran nmap -sV 192.168.20.100.	Discover services on the DMZ target.	80/tcp open; Apache httpd 2.4.52 (Ubuntu).	Validated
AL-22	Kali scan	Ran sudo nmap -sS -sV -O -T4 -p- 192.168.20.100.	Generate stronger controlled reconnaissance traffic.	Full-port SYN/version/OS scan completed.	Completed
AL-23	Suricata alert	Reviewed Suricata alerts after the scan.	Check whether IDS detected the generated activity.	Priority 1; Web Application Attack; 192.168.10.101 -> 192.168.20.100:80.	Validated
AL-24	Wazuh	Started the single-node Docker deployment and checked compose status.	Bring up SIEM manager, indexer and dashboard.	All three containers showed Up.	Completed
AL-25	Wazuh config	Added UDP 514 syslog <remote> block allowing 192.168.10.1.	Allow pfSense to send firewall syslog events.	Running ossec.conf later showed the block.	Completed
AL-26	pfSense syslog	Configured remote logging to 192.168.10.101:514 through the web GUI.	Send pfSense logs to the Wazuh host.	Initial automatic capture showed no UDP 514 packets.	Issue
AL-27	Network test	Ran sudo tcpdump -ni any udp port 514 on Kali.	Check whether syslog packets reached the host.	Initial captures: 0 packets.	Issue
AL-28	Isolation test	Used pfSense logger to send a direct remote syslog test to 192.168.10.101:514.	Separate network-path failure from automatic logging configuration.	1 packet captured from 192.168.10.1 to 192.168.10.101:514.	Validated
AL-29	pfSense GUI	Disabled remote logging, saved, then re-enabled it with one server, IPv4, Default (any), and Everything for the test.	Reset and cleanly save the automatic remote logging state.	Automatic syslog packets began arriving.	Completed

ID	Phase	Action / command	Reason	Evidence / result detail	Status
AL-30	Packet capture	Captured automatic UDP 514 syslog with tcpdump.	Prove pfSense remote logging now worked without the direct logger test.	66 packets captured; 192.168.10.1:514 -> 192.168.10.101:514 and Docker path.	Validated
AL-31	Wazuh config	Read the running manager configuration.	Confirm the container loaded the syslog listener settings.	connection=syslog, port=514, protocol=udp, allowed- ips=192.168.10.1.	Validated
AL-32	Wazuh listener	Checked /proc/net/udp for hexadecimal port 0202.	Verify a UDP 514 socket existed even though ss was unavailable.	00000000:0202 was present.	Validated
AL-33	Raw log capture	Ran sudo tcpdump -A -ni eth1 udp port 514 -c 1 and generated a blocked DMZ ping.	Read the actual syslog payload.	filterlog payload contained block, 192.168.20.100 and 192.168.10.100.	Validated
AL-34	Wazuh alert check	Searched alerts.log for filterlog, 192.168.20.100 and a matching pfSense alert.	Check end-to-end SIEM alert generation.	No matching pfSense alert was confirmed.	Not confirmed
AL-35	Documentation	Recorded the Wazuh result as transport-validated but alert-correlation incomplete.	Avoid overstating the project outcome.	Limitation documented and future decoder/rule tuning identified.	Completed

Appendix C - Troubleshooting Decision Log

This decision log shows how I narrowed down problems instead of changing several settings at once. The most important lesson was to test the layer closest to the symptom and only move deeper after that layer was proven.

ID	Symptom	Initial hypothesis	Diagnostic action	Finding	Decision / fix	Final result	Status
TD-01	Ubuntu had no correct DMZ IPv4 address.	VirtualBox adapter or Netplan could be wrong.	Verified DMZ_NET attachment and interface name enp0s3; inspected Netplan state.	Interface existed, but DHCP configuration was not correct/persistent.	Set DHCPv4 for enp0s3 and applied Netplan.	Ubuntu received 192.168.20.100.	Closed
TD-02	Apache service needed proof before firewall testing.	A failed browser test could be a service problem rather than a firewall problem.	Installed Apache2 and checked systemctl status.	Service was active (running).	Tested from Windows at http://192.168.20.100.	Apache default page loaded.	Closed
TD-03	DMZ could not ping LAN after block rule.	Could be pfSense policy, Windows firewall or routing.	Checked pfSense Firewall Logs immediately after the test.	pfSense logged the exact source and destination under the block rule.	Used firewall log as authoritative policy evidence.	Segmentation control confirmed.	Closed
TD-04	Suricata initially showed generic protocol alerts.	Traffic did not yet clearly represent the intended reconnaissance test.	Ran a stronger, controlled Nmap scan from Kali to the DMZ server.	IDS alerts then showed Kali 192.168.10.101 -> Ubuntu 192.168.20.100:80 with Priority 1.	Compared scan source/destination with alert source/destination.	Detection result confirmed.	Closed
TD-05	pfSense remote logging was configured, but tcpdump showed zero UDP 514 packets.	Network path, pfSense sending, or Wazuh could be the problem.	Sent a direct logger message from pfSense while tcpdump listened on Kali.	The direct packet reached 192.168.10.101:514.	Focused on pfSense automatic remote logging configuration, not the network path.	Direct path validated.	Closed
TD-06	Automatic pfSense syslog still did not appear after initial GUI setup.	Saved remote logging state may not have been applied cleanly.	Disabled remote logging, saved, re-enabled it with one server entry, IPv4 and Everything for testing.	Automatic UDP 514 packets began arriving.	Repeated tcpdump and captured 66 packets.	Automatic remote logging confirmed.	Closed
TD-07	Wazuh alert searches were blank although syslog traffic was visible.	Transport could work while decoding/rule matching remained incomplete.	Verified running ossec.conf, checked UDP 514 socket in /proc/net/udp, and captured the raw filterlog payload.	Listener and transport were present; the payload contained the expected firewall block.	Stopped claiming an end-to-end Wazuh alert and documented decoder/rule tuning as future work.	Transport validated; alert correlation incomplete.	Open

Appendix D - Risk Register and Remediation Plan

The risk register below separates risks that were addressed in the lab from items that remain open. Severity is a practical lab assessment, not a formal enterprise risk score.

ID	Risk / finding	Severity	Current control / evidence	Residual risk	Status	Recommended action
F-01	Unrestricted DMZ-to-LAN movement could expose internal hosts if the DMZ server were compromised.	High	DMZ-to-LAN block rule placed above allow rules and logging enabled.	Low	Effective	Retain the block rule and review logs for denied traffic.
F-02	pfSense displayed a warning that the admin account password was set to the default value.	High	No evidence in this project that the default password was changed.	High	Open	Change the pfSense admin password in User Manager and record the hardening step.
F-03	pfSense firewall events reached Wazuh transport, but a meaningful Wazuh dashboard alert was not confirmed.	Medium	UDP 514 listener and raw filterlog delivery were validated.	Medium	Open	Use wazuh-logtest with the captured filterlog event, confirm the decoder, and create/tune a local rule.
F-04	The pfSense-to-Wazuh syslog path uses UDP and does not provide confidentiality or delivery assurance by itself.	Medium	Traffic remained inside the isolated lab network.	Low in lab / Medium in production	Accepted for lab	For production, place logging on a protected management network or use a relay/collector design that can forward securely.
F-05	Suricata was operated in IDS mode, so matching traffic generated alerts but was not automatically blocked.	Medium	pfSense firewall policy still enforced segmentation; Suricata was used for detection.	Low for this test	By design	Tune and validate signatures before considering IPS/blocking mode.
F-06	The Kali test system and Wazuh Docker stack share the same VM, which mixes attacker/test and monitoring roles.	Medium	Lab isolation and known test traffic reduced impact.	Medium	Open architecture item	Separate the SIEM server from the security testing host in a future version of the lab.

Priority remediation

The first hardening action should be changing the pfSense admin password because the GUI warning showed the default value was still configured. The next technical improvement should be Wazuh decoder/rule tuning so the validated filterlog event becomes a searchable and meaningful SIEM alert.

Appendix E - Evidence Matrix

This matrix links the main project claims to the test and screenshot used to support them. I included it to make the report easier to review on GitHub or during an interview.

Claim	Validation method	Evidence	Status
WAN/LAN/DMZ interfaces were assigned correctly.	Review pfSense Interface Assignments.	Figure 1	Completed
Windows joined the LAN through pfSense DHCP.	Check IP address and default gateway.	Figure 2	Completed
LAN client had gateway and outbound connectivity.	Ping 192.168.10.1 and 8.8.8.8.	Figure 3	Completed
Ubuntu hosted a reachable DMZ web service.	Open http://192.168.20.100 from Windows.	Figure 4	Completed
DMZ policy blocks traffic to the LAN.	Ping Windows from Ubuntu and review firewall logs.	Figures 5-7	Completed
Suricata rules were available and updated.	Review Suricata rule update result.	Figure 8	Completed
Kali discovered the Apache service.	Run Nmap service detection against 192.168.20.100.	Figure 9	Completed
Suricata detected the controlled scan traffic.	Review IDS alerts after Nmap scan.	Figure 10	Completed
Wazuh single-node services were running.	Run docker compose ps.	Figure 11	Completed
Wazuh manager configuration contains UDP 514 syslog listener.	Read running ossec.conf inside manager container.	Figure 12	Completed
Automatic pfSense syslog reached the Wazuh host/Docker path.	Run tcpdump on UDP 514.	Figure 13	Validated
The captured syslog event was a real pfSense firewall block.	Capture one UDP 514 packet in ASCII mode.	Figure 14	Validated
pfSense firewall event became a confirmed Wazuh dashboard alert.	Search alerts.log/dashboard for matching event.	No positive evidence	Not confirmed

Final assessment

The project is complete as a firewall, DMZ segmentation and IDS portfolio lab. The SIEM integration is honestly described as transport-validated rather than fully correlated at the dashboard alert layer. This distinction makes the report stronger because every claim can be traced to evidence.

Appendix F - Interview Explanation

“I built a multi-VM cybersecurity lab with pfSense separating a LAN and DMZ. I hosted Apache in the DMZ, wrote firewall rules to stop the DMZ from initiating traffic into the LAN, and proved the block using both packet loss and pfSense filter logs. I used Kali and Nmap to generate controlled reconnaissance traffic, which Suricata detected as Priority 1 alerts. I also integrated pfSense remote syslog with my Wazuh Docker environment and used tcpdump to prove the actual filterlog events reached the Wazuh path over UDP 514. The Wazuh dashboard alert correlation still needs decoder or local-rule tuning, and I documented that limitation instead of claiming it was fully complete.”